

Linked List Operations

Contents

1	Introduction	2
2	What is Singly Linked List?	2
3	Example of Singly Linked List	2
4	Representation of Singly Linked List	3
5	Singly Linked List Operations in Data Structure	3
5.1	Insertion Operations	3
5.2	Deletion Operations	4
5.3	Display Operation	5
5.4	Search Operation	5
6	Advantages of Singly Linked Lists	6
7	Disadvantages of Singly Linked Lists	6
8	Practical Applications of Singly Linked Lists	6
9	Source Code	7
9.1	Singly Linked List Implementation in Java	7
9.2	Singly Linked List Implementation in Python	8
9.3	Singly Linked List Implementation in C Language	9
9.4	Singly Linked List Implementation in C++	10

1 Introduction

A singly linked list is a basic type of data structure that includes a series of connected nodes. Each node stores a piece of data and a link to the next node in the sequence.

This simple arrangement makes singly linked lists a popular choice for organizing data where easy insertion and deletion of elements are required without the overhead of pre-allocating space, as seen in arrays.

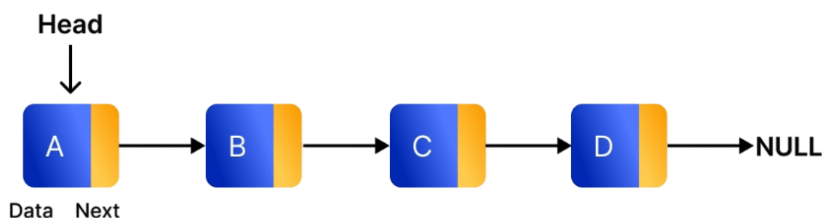
For instance, a common application is creating a dynamic queue of tasks where tasks are continuously added and processed.

Here, we will learn everything about singly linked lists, showing how they work and are used in various DSA and programming scenarios.

2 What is Singly Linked List?

A singly linked list in data structures is essentially a series of connected elements where each element, known as a node, contains a piece of data and a reference to the next node in the sequence.

This structure allows for easy and efficient addition or removal of elements without rearranging the entire data structure, making it a flexible choice for many applications.



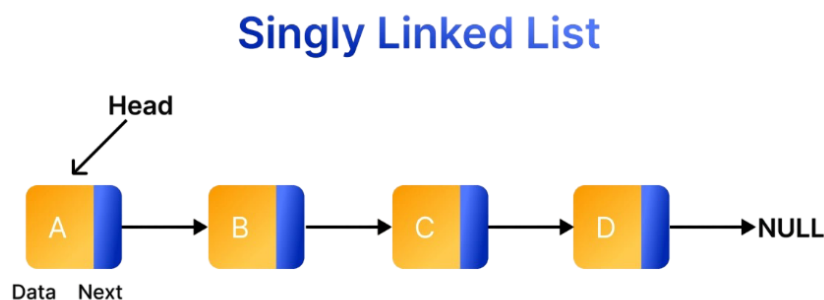
3 Example of Singly Linked List

Consider an application like a dynamic to-do list where users can add and remove tasks. Here, a singly linked list allows new tasks to be added or old tasks to be deleted without the need to shift other elements.

This flexibility and efficiency make singly linked lists a popular choice for implementing other foundational data structures, such as **stacks** and **queues**, and for applications like memory management, where the allocation and deallocation of memory happen frequently and in varying sizes.

4 Representation of Singly Linked List

- **Node:** The fundamental part of a singly linked list. Each node consists of two components:
 - **Data:** This part of the node stores the actual data that the list is meant to hold. It can be any type of data—numbers, characters, or even more complex data structures.
 - **Next:** This is a pointer (or reference) to the next node in the list. It's how the list maintains its sequence, by linking each node to the subsequent one.



- **Head:** The first node in a linked list is called the head. It is the entry point to the list and used as a reference point to traverse it.
- **Null:** The last node of a linked list, which points to null, indicating the end of the list.
- **Functionality of Singly Linked List:** The list starts with a head pointer, which points to the first node in the list. The last node in the list points to null, indicating that there are no more nodes after it. This null marking the end of the list is crucial - it tells any process or algorithm when to stop iterating through the list.

5 Singly Linked List Operations in Data Structure

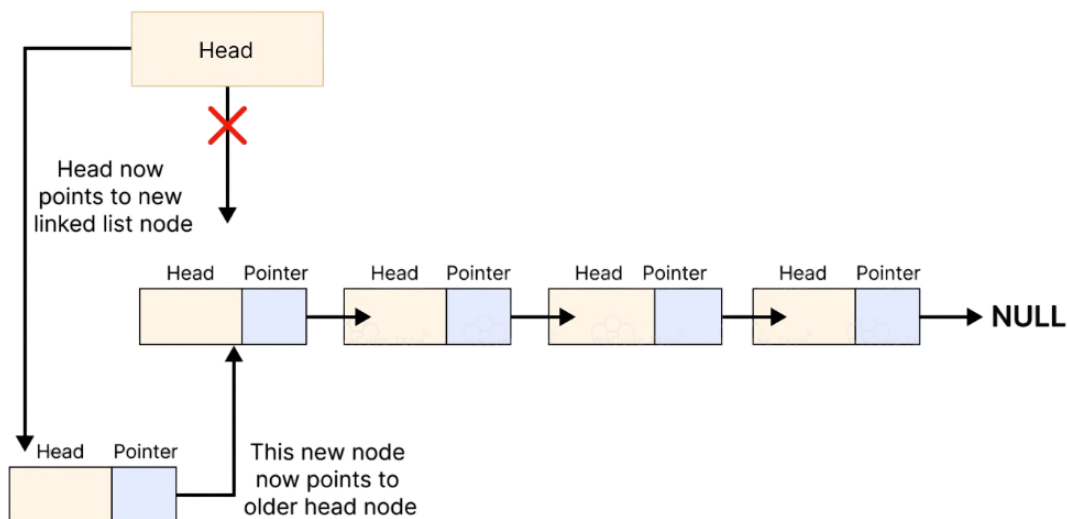
Following are the primary operations you can perform on a singly linked list.

5.1 Insertion Operations

- **Insertion at the Beginning:** Also known as "Insertion at Head." This operation involves adding a new node right at the start of the list. The new node then becomes the head of the list. This operation is quick because it simply includes pointing the new node to the current head of the list and then updating the head to this new node, making it a constant time operation, $O(1)$.

- **Insertion at the End:** Known as "Insertion at Tail." This requires traversing the entire list to find the last node and then adding the new node after this. The new node's next pointer is set to null, indicating the end of the list. Since you need to traverse the entire list, this operation has a time complexity of $O(n)$, where n is the number of nodes in the list.
- **Insertion after a Given Node:** If you need to insert a new node after a specified node, you connect the new node to the list by adjusting the pointers. The new node points to the next node of the current node, and the current node's next pointer is updated to point to the new node. This operation generally requires $O(n)$ in the worst case because you might need to traverse the list to find the specified node.

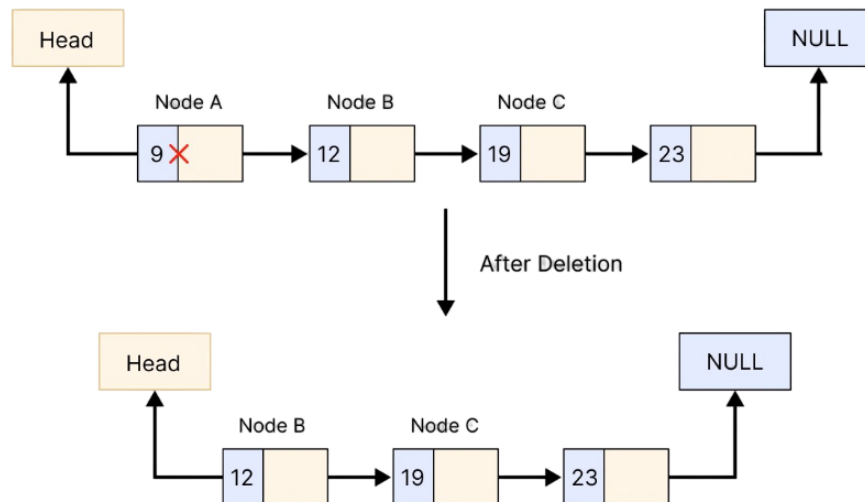
Insertion at Beginning



5.2 Deletion Operations

- **Deletion at the Beginning:** Removing the first node (head) of the list can be done by simply updating the head to point to the second node. This is also a constant time operation, $O(1)$.
- **Deletion from the Middle or Specific Node:** To delete a node after a specific node, you update the next pointer of the preceding node to skip the node to be deleted and point to the following node. Depending on the position of the node to delete, this can also involve traversing the list, making it $O(n)$.
- **Deletion at the End:** Deleting the last node requires traversing the list to find the second last node and updating its next pointer to null. This operation takes $O(n)$ time as well.

Deletion At Beginning

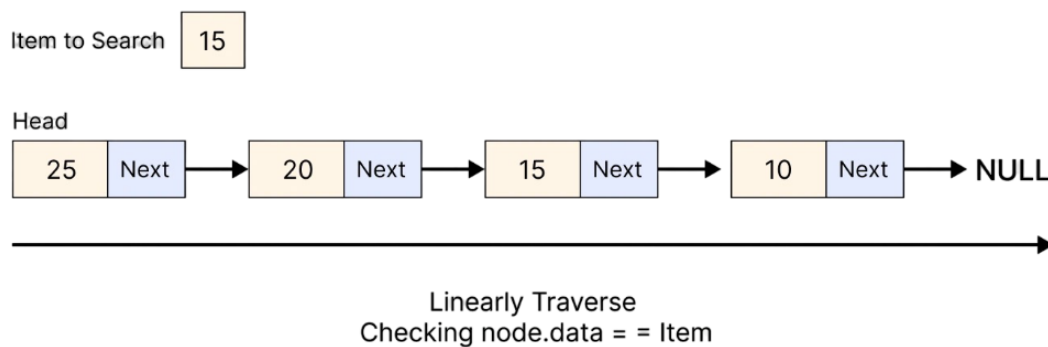


5.3 Display Operation

To display or traverse the singly linked list, start from the head and move through each node until you reach the end. Each node's data is accessed as you traverse, and this operation has a time complexity of $O(n)$.

5.4 Search Operation

Searching involves traversing the list from the head and checking each node's data against what you are searching for. This operation also has a time complexity of $O(n)$ in the worst case.



6 Advantages of Singly Linked Lists

- Singly linked lists in data structures can grow and shrink during runtime as needed without a predefined size.
- They only allocate memory for nodes that are actually in use, reducing memory waste compared to pre-allocated data structures like arrays.
- Adding or removing nodes doesn't require shifting elements, which can be a costly operation in arrays. This is particularly beneficial at the beginning of the list.
- Unlike arrays, linked lists don't reserve memory in advance, which can be more efficient for certain types of applications where the size of the data structure fluctuates.

7 Disadvantages of Singly Linked Lists

- Accessing an element in a singly linked list requires traversal from the head to the point of interest, which can be time-consuming.
- Each node in a singly linked list requires additional memory for the pointer alongside the actual data.
- Implementing a singly linked list is more complex than using an array, particularly when it comes to handling pointers, which can introduce bugs like memory leaks.

8 Practical Applications of Singly Linked Lists

Singly linked lists are versatile data structures that find practical applications across various aspects of computing, from basic data structure implementations to complex system operations.

1. Implementing Stacks and Queues

Singly linked lists are ideal for implementing stacks and queues due to their dynamic nature and efficient insertion and deletion operations.

- **Stacks:** Stacks follow a Last In, First Out (LIFO) principle. With a singly linked list, you can add (push) or remove (pop) elements from the head of the list, which ensures that the most recently added element is always the first to be removed. This operation is efficient, with $O(1)$ complexity for both push and pop actions.
- **Queues:** Queues operate on a First In, First Out (FIFO) basis. You can efficiently enqueue (add) elements at the end of a singly linked list and dequeue (remove) them from the head. This setup maintains the order in which elements are added and removed, making it suitable for scenarios like processing tasks, handling operations in a service center, or managing print jobs.

2. Memory Management

In systems programming, memory management is crucial, and singly linked lists can play a significant role.

Singly linked lists are used to track free memory blocks in dynamic memory allocation. Each node in the list can represent a block of memory, with its size and status (free or occupied).

As processes request and release memory, the list is updated to merge or split memory blocks efficiently, which helps in avoiding fragmentation and optimizes memory usage.

3. Undo Functionality in Applications

Many applications require the ability to undo actions performed by the user, and singly linked lists are a natural fit for implementing this functionality.

Each action that can be undone is stored as a node in a singly linked list. For example, in text editors or graphic design software, each edit action (like text addition, deletion, or image adjustment) can be a node in the list.

To undo an action, the program simply traverses back to the previous node, restoring the state before the last action was taken. This method is simple to implement and scales well with the number of actions.

9 Source Code

9.1 Singly Linked List Implementation in Java



```
1 public class Node {
    public int data;
3    public Node next;

5    public Node(int data) {
        this.data = data;
7        this.next = null;
    }
9 }

11 public class SinglyLinkedList {
    private Node head;
13
    public void append(int data) {
15        Node newNode = new Node(data);
        if (head == null) {
17            head = newNode;
            return;
19        }

21        Node last = head;
```



```

23     while (last.next != null) {
24         last = last.next;
25     }
26     last.next = newNode;
27 }
28
29 public void printList() {
30     Node current = head;
31     while (current != null) {
32         System.out.print(current.data + " -> ");
33         current = current.next;
34     }
35     System.out.println("None");
36 }
37
38 public static void main(String[] args) {
39     SinglyLinkedList linkedList = new SinglyLinkedList();
40     linkedList.append(1);
41     linkedList.append(2);
42     linkedList.append(3);
43     linkedList.printList(); // Output: 1 -> 2 -> 3 -> None
44 }

```

9.2 Singly Linked List Implementation in Python



```

1 class Node:
2     def __init__(self, data):
3         self.data = data
4         self.next = None
5
6 class SinglyLinkedList:
7     def __init__(self):
8         self.head = None
9
10    def append(self, data):
11        new_node = Node(data)
12        if not self.head:
13            self.head = new_node
14            return
15        last_node = self.head
16        while last_node.next:
17            last_node = last_node.next
18        last_node.next = new_node
19
20    def print_list(self):
21        current_node = self.head

```




```

22 while current_node:
    print(current_node.data, end=" -> ")
24     current_node = current_node.next
    print("None")
26
# Example Usage
28 linked_list = SinglyLinkedList()
    linked_list.append(1)
30 linked_list.append(2)
    linked_list.append(3)
32 linked_list.print_list() # Output: 1 -> 2 -> 3 -> None

```

9.3 Singly Linked List Implementation in C Language



```

#include <stdio.h>
2 #include <stdlib.h>

4 // Definition of a node
typedef struct Node {
6     int data;
    struct Node *next;
8 } Node;

10 // Function to create a new node
Node *createNode(int data) {
12     Node *newNode = (Node*) malloc(sizeof(Node));
    newNode->data = data;
14     newNode->next = NULL;
    return newNode;
16 }

18 // Function to append a node to the end of the linked list
void append(Node **head, int data) {
20     Node *newNode = createNode(data);
    if (*head == NULL) {
22         *head = newNode;
    } else {
24         Node *last = *head;
        while (last->next != NULL) {
26             last = last->next;
        }
28         last->next = newNode;
    }
30 }

32 // Function to print the linked list
void printList(Node *head) {

```



```
34 Node *current = head;
   while (current != NULL) {
36     printf("%d -> ", current->data);
       current = current->next;
38 }
   printf("NULL\n");
40 }

42 int main() {
   Node *head = NULL;
44 append(&head, 1);
   append(&head, 2);
46 append(&head, 3);
   printList(head); // Output: 1 -> 2 -> 3 -> NULL
48 return 0;
}
```

9.4 Singly Linked List Implementation in C++



```
1 #include <iostream>

3 using namespace std;

5 // Node class
   class Node {
7   public:
       int data;
9       Node *next;

11  public:
       Node(int data) {
13         this->data = data;
           this->next = nullptr;
15     }
   };

17 // SinglyLinkedList class
19 class SinglyLinkedList {
   public:
21     Node *head;

23  public:
       SinglyLinkedList() :
25         head(nullptr) {}

27     void append(int data) {
       Node *newNode = new Node(data);
```



```
29     if (head == nullptr) {
30         head = newNode;
31     } else {
32         Node *last = head;
33         while (last->next != nullptr) {
34             last = last->next;
35         }
36         last->next = newNode;
37     }
38 }
39
40 void printList() {
41     Node *current = head;
42     while (current != nullptr) {
43         cout << current->data << " -> ";
44         current = current->next;
45     }
46     cout << "NULL" << endl;
47 }
48 };
49
50 int main() {
51     SinglyLinkedList list;
52     list.append(1);
53     list.append(2);
54     list.append(3);
55     list.printList(); // Output: 1 -> 2 -> 3 -> NULL
56     return 0;
57 }
```